

Charlie Daubney  
Student ID  
100184278  
Bolton University

# **Object Oriented Programming**

## **Introduction**

The term Object-Oriented programming (OOP) refers to the concept that certain programming languages focus around “Objects” which can contain data in the form of fields (properties or attributes) and code, in the form of procedures (methods). OOP allows programmers to model real world entities and interactions through four main principles. Encapsulation, Inheritance, Abstraction, and polymorphism (Sebesta, 2016). This report will delve into each these principles in detail as well as their importance and practicality in real world situations within software engineering as well as any examples which may be found during research.

## **Encapsulation**

Encapsulation can be defined in several different ways but i will go with the most relevant to the project. Encapsulation is the process where variables and methods are put into a single class to hide the internal details. Looking at encapsulation and its practical use in industry will help to further understand the term. Encapsulation can be used to keep debugging simple, they are contained by themselves so they can be tested separately. It can also be used to keep all the more intricate data hidden from the user, and displaying only the important functionality information to the user. The code can also be adjusted freely and easily in the case of expanding the or downsizing the project without affecting the main code, it can be easily changed. For example, encapsulation allows for proper object integrity, if a user was trying to take money out of the bank they didnt have, the object integrity of encapsulation on the logic inside of the encapsulated getter would output a message letting the user know they cannot take that out as they do not have it. Encapsulation is vital to include in object oriented programming to ensure proper code integrity, simple debugging and modularity.

## **Inheritance**

Inheritance refers to child classes inheriting attributes or processes from the parent class. This is easiest described if somewhere in your code you have a class for Teacher, this class includes Name and Age, if another class is created for student ("Student" class) name and age can both be inherited from the Teacher t class as they apply to the student also. This is convenient for reusing attributes to simplify the amount of work and keep the code clean. Outside of simplicity inheritance provides a realistic model for the code as it's a hierarchy of classes which inherit from one another (Person>Teacher>Student), we see this in industry most commonly in android APK development as almost every screen you see on android phone is a subclass of the main class. Perhaps the most important point of inheritance is that it enables other concepts of OOP such as polymorphism and Abstraction where a subclass is needed in order for methods to start overriding classes, Abstraction is involved as it is important to abstract minor details whilst keeping around the important ones. Overall inheritance provides the coder with the ability to reuse attributes rather than creating new ones, open up to including more OOP concepts in their code, and implement real world hierarchies into their code for better understanding and performance.

## **Abstraction**

Abstraction can be defined as the key takeaways from implementing new classes and attributes into the code whilst weeding out the unimportant aspects. The example I'm going to use includes an animal class where Cats is an attribute, if we decide to create subclasses for animals based on the sounds they make, cat will have meow, which is an abstract subclass as it is an important detail which is specific to that animal and editing this will not affect the entire Animal class (Farrell, J. 2018). Abstraction is important because like the other concepts it is independent, it is easy to debug and will not interfere with other parts of the code if it is removed or changed. It simplifies code and minimises complexity and saves the need for several other subclasses for unnecessary details. Overall abstraction promotes simple and convenient code whilst minimising any unnecessary detail or dependencies.

## **Polymorphism**

Polymorphism allows for different classes to respond to the same function call in their own different ways as long as they share the same parent class. There are two main types of polymorphism which each have their own uses. Compile-Time polymorphism refers to the same method but with different parameters. And Run-Time polymorphism

which is the child classes own interpretation of a parent classes method. Polymorphism saves us the need for countless if else statements as it can just adapt a parent classes call and the software engineer can just tweak it to provide outputs to every type of input. It is flexible and scalable, the run time polymorphism can save time on having to mess around with the function calls from the parent class and can be edited separately as well. An industry example of polymorphism may be notification types in operating systems. Emergency notifications may take priority over email notifications but they are both members of the same parent class. Overall polymorphism is important for concision within code as well as scalability, flexibility, and working against time constraints as there isnt much need for long if else statements

## **OOP in industry**

As mentioned OOP is widely used in the industry of desktop applications, Phone operating systems, etc. It is used within large projects for its concision and effectiveness aswell as its scalability and ability to be faced with updates both large and small. It is important to bring up UML (Unified Modelling Language) as it is a key blueprint for creating object oriented based software before the coding process even begins. It helps to map the 4 key points this report has been based on before the actual coding process has even begun to ensure the project is scalable for future growth and maintainable for daily use and has been used in my own portfolio for this assignment.

## **Conclusion**

In conclusion this report allowed for me to delve into the importance of fully understanding OOP to improve the quality of my own work in object-oriented programming languages, i learnt the real world practicality and purpose of encapsulation, inheritance, abstraction and polymorphism and how they work together in large scale projects to maintain flexibility and scalability for future project extensions and updates.

## **References**

Oracle (n.d.). *Polymorphism (The Java™ Tutorials > Learning the Java Language > Interfaces and Inheritance)*. [online] docs.oracle.com. Available at:  
<https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html>.

Farrell, J. (2019). *Java programming*. Singapore: Cengage Learning Asia Pte Ltd.

DataCamp. (2025). *Java Encapsulation*. [online] Available at: <https://www.datacamp.com/doc/java/encapsulation#> [Accessed 18 May 2025].

geeksforgeeks (2017). *Encapsulation in Java - GeeksforGeeks*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/encapsulation-in-java/>.

DataCamp. (2025b). *Java Polymorphism*. [online] Available at: <https://www.datacamp.com/doc/java/polymorphism#> [Accessed 18 May 2025].

## **Bibliography**

**Computer Science (2020). *Fundamental Concepts of Object Oriented Programming. YouTube*. Available at: [https://www.youtube.com/watch?v=m\\_MOYyJpIjg](https://www.youtube.com/watch?v=m_MOYyJpIjg) [Accessed 5 Mar. 2021].**

**Keep On Coding (2020). *Object Oriented Programming - The Four Pillars of OOP. YouTube*. Available at: <https://www.youtube.com/watch?v=1ONhXmOuWP8> [Accessed 24 Oct. 2022].**